

Universidad de Guayaquil

Manual Tecnico
“Proyecto-Chat-Distribuidas”

Pionce Mendez Cristopher

Tabla de contenido

Descripcion General	3
Arquitectura del sistema	3
Tecnologias Utilizadas.....	4
Estructura del proyecto	4
Instalación y configuración.....	5
Funcionamiento del servidor.....	5
Funcionamiento del cliente	6
Comunicación mediante Sockets	6

Descripcion General

El sistema desarrollado es una aplicación de chat en tiempo real basada en arquitectura cliente-servidor. Permite el registro e inicio de sesión de usuarios, envío y recepción de mensajes instantáneos, visualización de usuarios conectados y almacenamiento persistente de mensajes en una base de datos PostgreSQL.

La comunicación en tiempo real se implementó mediante Socket.io y la autenticación mediante JSON Web Tokens (JWT).

Arquitectura del sistema

La solución está compuesta por tres capas principales:

Frontend

- React
- Vite
- Bootstrap
- Axios
- Socket.io Client

Backend

- Node.js
- Express.js
- Socket.io
- JWT
- Prisma ORM

Base de datos

- PostgreSQL (Neon)

Despliegue

- Frontend: Vercel
- Backend: Render
- Base de datos: Neon

Tecnologías Utilizadas

Tecnología	Función
React	Interfaz de usuario
Vite	Herramienta de desarrollo frontend
Bootstrap	Diseño responsivo
Axios	Consumo de API REST
Node.js	Entorno de ejecución
Express	Servidor web
Socket.io	Comunicación en tiempo real
JWT	Autenticación
Prisma ORM	Acceso a base de datos
PostgreSQL	Persistencia de datos
Git/GitHub	Control de versiones
Render	Despliegue backend
Vercel	Despliegue frontend
Neon	Base de datos en la nube

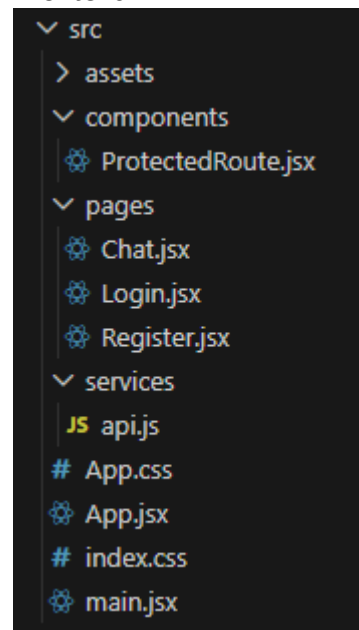
Estructura del proyecto

Backend

```

  src
  |
  |  config
  |  |  prisma.js
  |  |
  |  controllers
  |  |  authController.js
  |  |  messageController.js
  |  |  userController.js
  |  |
  |  middleware
  |  |  authMiddleware.js
  |  |
  |  routes
  |  |  authRoutes.js
  |  |  messageRoutes.js
  |  |  userRoutes.js
  |  |
  |  > sockets
  |  |
  |  JS server.js
  |
  |  .env
  |  .gitignore
  |  package-lock.json
  |  package.json
  |  prisma.config.ts
  |  testSocket.js
```

Frontend



Instalación y configuración

Backend

Instalar dependencias:

- `npm install`

Configurar variables de entorno:

- `DATABASE_URL=postgresql://...`
`JWT_SECRET=miclave`
`PORT=3000`

Ejecutar servidor:

- `npm run dev`

Frontend

Instalar dependencias:

- `npm install`

Ejecutar aplicación:

- `npm run dev`

Funcionamiento del servidor

El servidor Express expone rutas REST para:

Autenticación

POST `/api/auth/register`

POST `/api/auth/login`

Usuarios

- GET /api/users

Mensajes

- GET /api/messages

Los mensajes son almacenados mediante Prisma en PostgreSQL.

Funcionamiento del cliente

El cliente React permite:

- Registro de usuarios.
- Inicio de sesión.
- Almacenamiento temporal del token JWT.
- Conexión al servidor Socket.io.
- Envío y recepción de mensajes.
- Visualización de usuarios conectados.
- Cierre de sesión.

Comunicación mediante Sockets

Evento Join

Permite registrar un usuario conectado.

- socket.emit('join')

Evento Message

Permite enviar mensajes al servidor

- socket.emit('message')

Evento Users Online

Actualiza la lista de usuarios conectados:

- socket.on('users_online')

Evento Disconnect

Se ejecuta cuando un usuario abandona la aplicación.